15B17CI371 – Data Structures Lab
ODD2026
Week 2, Practice Lab [CO: C270.1]

Instructions:

1. **All students must save their programs using the following naming convention:** (Enroll. No_W2) with docx or pdf formats. For each question, mention the question, its solution C++ program, and its execution output.
2. **Students are advised to write code for all questions** to ensure thorough practice of all related topics.

Code of Conduct:

1. **Avoid using mobile phones, earphones, or watching YouTube videos during lab hours.** Defaulters will receive zero marks in evaluations.
2. **AI Tool Usage Policy:** Zero marks will be awarded for the sole or extensive use of AI tools.

Do not's	Dos
Do not solely/extensively use AI tools to draft your abstract, synopsis, report, etc.	You may use AI tools for spelling or grammar checking as you write.
Do not use AI tool to summarize or complete your project code	You may use AI to search relevant materials, and do programming by yourself.
Do not use AI tool to complete, analyse or rectify errors in your project code.	You may ask your mentor to help with your code if you get stuck.

Concepts: Linked Lists

Practice Lab Questions:

Q1. You are given an empty singly linked list. Assume that this list contains only whole numbers. Write functions to

a. Insert 'n' number of data in the singly linked list. Insert from the head.
Practice this question using Virtual Lab (https://ds1-iiith.vlabs.ac.in/exp/linked-list/singly-linked list/sllexercise.html)

b. Find the total number of nodes in the linked list, and give their average.

c. Print first 'm' data from the linked list. Assume that 'm' is less than 'n'.
Example:
Input: Linked list: {1, 3, -9, 45, 2, 3, 56, 100, -67} m=4
Output: {1, 3, -9, 45, 2}
Example:
Input: Linked list: {1, 3, -9, 45, 2, 3, 56, 100, -67} m=10
Output: Incorrect value of m

d. Find the middle element of the linked list and check if it's odd or even. Print an appropriate output.
Example:
Input: Linked list: {1, 3, -9, 45, 2, 3, 56, 100, -67}
Output: 3 is odd

e. Find the 'l' number from the end of the list.
Example:
Input: Linked list: {1, 3, -9, 45, 2, 3, 56, 100, -67} l=3
Output: {56, 100, -67}

f. Find if a given number exists in the list. If it does, write a function to delete it.
Example:

Input: Linked list: {1, 3,-9, 45, 2, 3, 56, 100, -67}
Number to be found: 45
Output: 45 exists in the original list
Final list: {1, 3, -9, 2, 3, 56, 100, -67}
If the value exists multiple times, delete only the first instance.

g. Interchange a pair of values with another given pair in the linked list.
Example:
Input: Linked list: {1, 3,-9, 45, 2, 3, 56, 100, -67}
Pairs to be exchanged: {1,3} with {56,100}
Output: {56, 100, -9, 45, 2, 3, 1, 3, -67}
Hint: first check if the pair exists and then apply the interchange function. If multiple or duplicate pairs are found, consider the first instance of the pair.

h. Check whether a given sub-list exists in the given linked list. If it exists, give its position (i.e., the starting position of the sub-list in the master linked list). **Example:**
Input: Linked list: {1, 3,-9, 45, 2, 3, 56, 100, -67}
Sub-list to be: {3, -9, 45}
Output: Exists at position 2.
{1, 3,-9, 45, 2, 3, 56, 100, -67}
Assumption: consider only the first occurrence of the sub-list.

i. Reverse a sub-list in the given linked list.
Example:
Input: Linked list: {1, 3,-9, 45, 2, 3, 56, 100, -67}
Sub-list to be reversed: {3, -9, 45}
Output: {1, 45,-9, 3, 2, 3, 56, 100, -67}
Assume that the user inputs the sub-list is found in the master linked list.

Q2. Assume that you have a linked list that can contain strings, i.e., each node can contain a string. Write a function to:

a. Print all the nodes in the linked list
b. Print all the strings (node values) that start with a particular alphabet.
c. Find if a given string exists in the linked list or not. Give appropriate output message.
d. Find the string with maximum length.
e. Find if a node contains "xyz" as a sub-string or not. Give appropriate output message.
f. Interchange the strings given in the positions p1, p2. These positions are user input. Check conditions that both p1 and p2 position exist in the given linked list, eg: suppose that your linked list consists of 4 strings only, and if user given p1=7, p2 = 10, then error message must be generated. g. Delete a given node (either by value or by position).

Q3. Create a link list of users supplied ten characters to store a name. Create a second link list of the same type of user supplied five characters. Now using a function **remove()**, traverse first link list and if any three consecutive characters of second link list appears as consecutive characters of first link list, remove those from first link list.

Q4. Implement a circular linked list that can contain integer elements. Add functions to:

a. Insert elements.
b. Print elements
c. Count the number of elements
d. Find if any element has a negative value.
e. Find the number of nodes having a value greater than 15.
f. Delete a particular element from the list.
g. Update the value of a particular element.
h. Insert a value at a given position.
i. Delete all nodes that have a prime number as their value.
j. Remove all the nodes from the list that contain Fibonacci data values.

You can practice a part of the above question using Virtual Lab facility. https://ds1-iiith.vlabs.ac.in/exp/linked-list/circular-linked list/cllpractice.html

Q5. Create an empty doubly linked list to store integers. Perform the following by writing appropriate functions to:

a. Insert and print elements of the list.
b. Traverse all nodes and check if the value is divisible by a number 'm'.
c. Delete all the nodes from the list that are greater than the given value 'x'. d. Find the number of elements between two duplicate values.

Example:

Input:
Doubly Linked list: {1, 3, -9, 45, 2, -56, 3, 56, 100, -67, 3, 3}
Duplicate element: 3
Output: No. of elements between a pair of '3' = 4.
Assumption: You are considering only the first instance of duplicity, i.e., between {3, -9, 45, 2, -56, 3} and not for instances like {3, 56, 100, -67, 3} nor
for {3, 3} or any others.

You can practice a part of the above question using Virtual Lab facility. https://ds1-iiith.vlabs.ac.in/exp/linked-list/doubly-linked list/dllexercise.html

Q6. Given a doubly linked list of any number of nodes, write a function **ExtremeSwap()**, which will swap values of the node at extreme pairs. For e.g., if the node values of a doubly linked list are:

1 2 3 4 5 6 7 8
After first call, values will be 8 2 3 4 5 6 7 1
After second call, values will be 8 7 3 4 5 6 2 1
And finally, function will stop after fourth call, and the values will be 8 7 6 5 4 3 2 1

Q7. Write a program to implement addition of two polynomials. Each node must contain the value of the coefficient as well as its power as data components.

Q8. Write a program to implement multiplication of two polynomials. Each node must contain the value of the coefficient as well as its power as data components. Take care of law of exponent multiplication.

Q9. Create a singly linked list of integers. Write a function to rotate the linked list to the left by K positions.

Example 1

Input:
Linked List: {10, 20, 30, 40, 50, 60}
K = 2
Output: {30, 40, 50, 60, 10, 20}
Example 2
Input:
Linked List: {5, 8, 12, 15, 20}
K = 4
Output: {20, 5, 8, 12, 15}

Q10. Create a sorted singly linked list of integers. Write a function to remove all consecutive duplicate nodes, keeping only one occurrence of each element.

Example 1
Input:
Linked List: {1, 1, 2, 3, 3, 3, 5, 5, 8}
Output: {1, 2, 3, 5, 8}
Example 2
Input:
Linked List: {4, 4, 4, 4, 4}
Output: {4}

You can practice questions on Linked lists from Virtual Lab using links: https://ds1-iiith.vlabs.ac.in/exp/poly-arithmetic/index.html

Certain relevant/handpicked Questions for interviews:

- Intersection of two Sorted Linked Lists
- Split a Circular Linked List into two halves
- Find pairs with given sum in doubly linked list
- Remove duplicates from an unsorted doubly linked list
- Intersection point of two Linked Lists
- Add two numbers represented by Linked lists
- Rearrange a given linked list in-place
- Reverse Alternate K Nodes in a Singly Linked List
- Reverse a Linked List in groups of given size
- Merge k Sorted Linked Lists
- Flattening a Linked List